Duoc UC – Sede Maipú

Capstone 2026 – Asignatura APT122

Evento Cultural

Documentación Técnica del Proyecto

Plataforma chilena de eventos culturales

Integrante	Rol principal
Roberto Riffo	Líder de proyecto / Frontend / UX
José Miguel Moncada	Base de datos / QA
David Herrera	Backend e integraciones

Roles según el design brief aprobado en Fase 1 (*design_brief_capstone.docx*).

Stack del sistema: Python (scraper con conectores, NVIDIA NIM y checkpoints SQLite), Supabase/PostgreSQL con Row Level Security, Edge Functions (`admin-account`), Angular 22 + Ionic 9 (web y móvil, catálogo real y autenticación directa vía `@supabase/supabase-js`), Leaflet + OpenStreetMap (mapa interactivo con pines georreferenciados) y GitHub Actions para automatización.

Versión 1.1

10 de octubre de 2026

Documento elaborado exclusivamente con evidencia verificable de las Fases 1 y 2.
Las omisiones se declaran como **pendientes del equipo** dentro de este mismo documento.

Índice

1. Introducción y alcance **3**
- 1.1. Estructura del sistema . . . 3

2. Arquitectura general del sistema **3**
- 2.1. Visión general . . . 3
- 2.2. Comunicación entre servicios . . . 4
- 2.3. Estado de la integración frontend–base de datos (actualizado 10-10-2026) . . . 4

3. Arquitectura del scraper **5**
- 3.1. Modelo general . . . 5
- 3.2. Conectores y fuentes . . . 6
- 3.3. Enriquecimiento selectivo con NVIDIA NIM . . . 7
- 3.4. Pipeline reanudable y concurrencia . . . 7
- 3.5. Patrones de diseño aplicados . . . 7
- 3.6. Pendientes de la arquitectura del scraper . . . 7

4. Arquitectura del frontend **8**
- 4.1. Stack real . . . 8
- 4.2. Estructura y navegación . . . 8
- 4.3. Filtros y funcionalidad . . . 10
- 4.4. Estrategia de despliegue . . . 10
- 4.5. Pendientes del frontend . . . 10

5. Modelo de datos (Supabase / PostgreSQL) **10**
- 5.1. Fuente del modelo . . . 10
- 5.2. Tablas y finalidad . . . 11
- 5.3. Seguridad (Row Level Security) y ciclo de carga . . . 12
- 5.4. Limitaciones declaradas . . . 12

6. Diagramas UML **12**
- 6.1. Casos de uso . . . 13
- 6.2. Secuencia de la funcionalidad principal . . . 14
- 6.3. Componentes y clases . . . 15
- 6.4. Pendientes de UML . . . 15

7. Requisitos no funcionales **15**
- 7.1. Requisitos definidos en Fase 1 (MoSCoW) . . . 15
- 7.2. Estado real por requisito . . . 15
- 7.3. Requisitos declarados y no implementados . . . 17
- 7.4. Limitaciones técnicas del desarrollo . . . 17

8. Pruebas **18**
- 8.1. Pruebas existentes del scraper . . . 18
- 8.2. Pruebas existentes del frontend . . . 18
- 8.3. Pruebas pendientes . . . 18
- 8.4. Registro de resultados . . . 18

9. Docker: estado actual y plan **18**
- 9.1. Estado real verificado . . . 18
- 9.2. Plan de implementación (alternativa documentada, no adoptada) . . . 19

10. Innovación **20**
10.1. ¿Qué problema resuelve? 20
10.2. ¿Qué hace diferente a la solución? 20
10.3. ¿Qué valor agrega? 20
10.4. Modelo de sostenibilidad y proyección de negocio (propuesta futura; no implementada en el MVP) 21

11. Trazabilidad documental **22**

1. Introducción y alcance

Este documento consolida la documentación técnica formal del proyecto **Evento Cultural**, una plataforma chilena para compilar y difundir eventos culturales, comerciales y municipales. Fue elaborado a partir de la evidencia disponible en el propio repositorio: el código y la documentación interna del scraper (`Scraper de eventos culturales.rar`), el código y la documentación del frontend (`evento-cultural.rar`), el modelo de base de datos (DBML, migraciones documentadas y diccionario de tablas), los requerimientos priorizados en Fase 1 (matriz MoSCoW) y el diagrama de arquitectura de Fase 2.

Convención de honestidad documental: este documento distingue explícitamente entre lo *implementado* (respaldado por código), lo *parcial* (respaldado solo en parte) y lo *pendiente* (información humana o trabajo futuro). Nada se afirma como terminado sin respaldo; los puntos abiertos se marcan con **Pendiente de completar por el equipo.**

1.1. Estructura del sistema

El sistema se compone de tres bloques independientes que se comunican a través de la base de datos: el **scraper** (recolección y enriquecimiento de datos), la **base de datos** (catálogo canónico en Supabase/PostgreSQL) y el **frontend** (aplicación web y móvil), con **GitHub Actions** como automatización de la ejecución del scraper (ver Figura 1 en la Sección 2).

2. Arquitectura general del sistema

2.1. Visión general

El sistema sigue una arquitectura de tres bloques desacoplados que se comunican a través de la base de datos, más un actor de automatización:

- **Scraper (Python):** recolecta eventos desde fuentes públicas chilenas, los normaliza y los enriquece selectivamente con NVIDIA NIM.
- **Base de datos (Supabase/PostgreSQL):** catálogo canónico con Row Level Security, bitácora de ejecuciones y evidencia de extracción.
- **Backend en TypeScript (misma arquitectura que el frontend):** el navegador consume Supabase directamente con `@supabase/supabase-js` (PostgREST y GoTrue) usando la clave pública, con Row Level Security gobernando el acceso. Las únicas operaciones con la clave de servicio (eliminar cuenta y registro pre-confirmado en desarrollo) viven en una **Edge Function** desplegada en Supabase (`admin-account`). Python fue retirado del flujo (migración del 10-10-2026).
- **Frontend (Angular 22 + Ionic 9):** aplicación web y móvil para descubrir, filtrar y publicar eventos; consume el catálogo y la sesión de Supabase directamente con supabase-js.
- **GitHub Actions:** automatiza la ejecución del scraper, ejecuta las pruebas y sincroniza la base de datos.

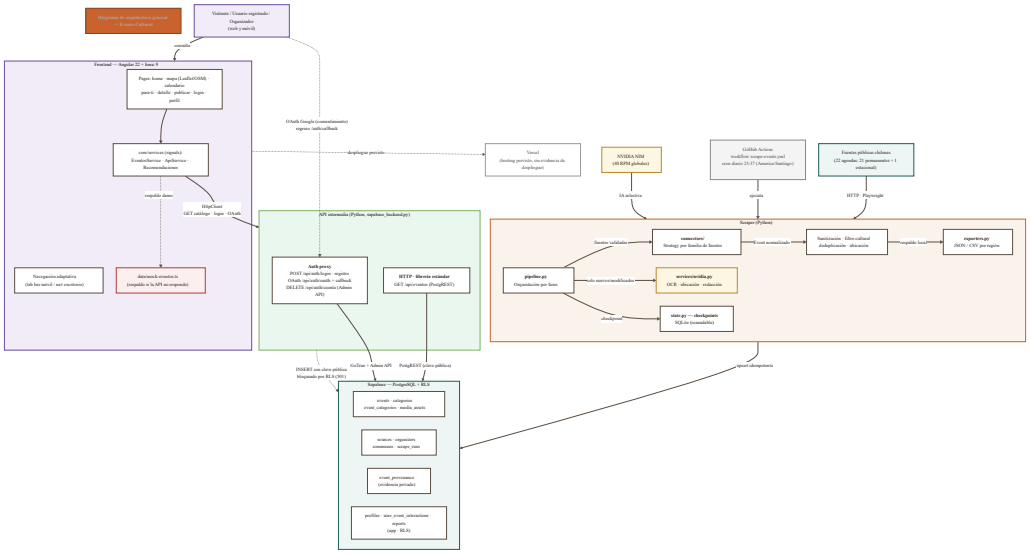


Figura 1: Arquitectura general del sistema (fuente editable: `Diagramas/01_arquitectura_general.mmd`, Mermaid).

2.2. Comunicación entre servicios

Cada fuente se procesa de manera independiente: si una falla, las demás continúan. Solo se publican eventos vigentes; la conciliación retira registros antiguos únicamente cuando la extracción fue completa y segura (diagrama de arquitectura de Fase 2).

2.3. Estado de la integración frontend–base de datos (actualizado 10-10-2026)

La integración está **implementada y funcional** en la última versión de la página web, con el backend migrado a TypeScript (supabase-js directo + Edge Function `admin-account`; sustituye a la API intermedia en Python, 10-10-2026):

Origen → Destino	Mecanismo	Detalle
GitHub Actions → Scraper	Ejecución de `scrape_events.py`	Fases separables (`--phase extract` / `--phase process`); estado restaurado por artefactos
Scraper → Fuentes	HTTP directo o Playwright	Caché condicional (ETag/Last-Modified), límites por dominio, respeto de `robots.txt` y `Retry-After`
Scraper → NVIDIA NIM	API de inferencia	Limitador global de 40 RPM con reintentos
Scraper → SQLite	Checkpoints por fuente y ficha	Reanudación de OCR/IA pendiente en futuras ejecuciones
Scraper → Supabase	API REST (upserts idempotentes)	Carga por lotes (`SUPABASE_BATCH_SIZE`); borrado de vencidos y conciliación configurables
Frontend → Supabase (directo)	`supabase-js` (PostgREST + GoTrue, clave pública, RLS)	**Implementado:** catálogo (`events` paginada más comunas/categorías), login y registro por contraseña, OAuth con regreso PKCE en `/auth/callback`
Frontend → Edge function admin-account	HTTP con JWT de sesión	Eliminar cuenta (Admin API) y registro preconfirmado solo en desarrollo; la clave de servicio nunca sale de Supabase

Cuadro 1: Comunicación entre servicios del sistema.

- `EventosService.cargarEventos()` pagina la tabla `events` publicada (`status=published`, no suprimidos) en bloques de 1.000 filas con `supabase-js`, junto con `communes` y `event_categories`, y traduce el esquema al modelo interno (snake_case → camelCase, incluyendo coordenadas para el mapa). Si la consulta falla, la interfaz lo indica y conserva los eventos de demostración.
- Autenticación real con Supabase Auth vía `supabase-js`: `signInWithPassword` y `signUp` (GoTrue directo), OAuth mediante `signInWithOAuth` con regreso PKCE en `/auth/callback` y `onAuthStateChange` para el estado reactivo de la sesión. La eliminación de cuenta pasa por la Edge Function `admin-account` con el JWT de sesión (la clave de servicio nunca sale de Supabase).
- Publicación de eventos: INSERT directo con el JWT del usuario; **pendiente la política RLS INSERT** por rol: hasta definirla, PostgREST responde 403 y la interfaz lo explica al organizador.

Pendiente de completar por el equipo: cronograma y responsables de la política INSERT por rol, el despliegue de la Edge Function admin-account y la activación de proveedores OAuth en Supabase Dashboard.

3. Arquitectura del scraper

3.1. Modelo general

Pipeline por fases con conectores intercambiables. Los ejecutables de la raíz (`scrape_events.py`, `add_source.py`, `validate_sources.py`) reciben la orden; el trabajo reside en el paquete `eventos/`. El flujo principal verificado en el entregable del scraper de Fase 2 es:

1. `SourceRepository` carga y valida los archivos de `sources/`.
2. El pipeline agrupa fuentes por dominio y las procesa con un pool acotado.

3. `ConnectorFactory` selecciona una Strategy según el campo `connector`; cada conector devuelve objetos `Event` con el mismo esquema.
4. `EventSanitizer` limpia los campos, limita descripciones y descarta vencidos.
5. Deduplicación por fuente y global, fijando el identificador estable.
6. `LocationNormalizer` aplica valores por defecto, reglas y, solo si falta información, IA.
7. Filtro regional y ubicación canónica.
8. `DatasetExporter` escribe el respaldo local (JSON/CSV por región).
9. Si Supabase está habilitado, `SupabaseExporter` sincroniza con upserts idempotentes y elimina vencidos remotos.

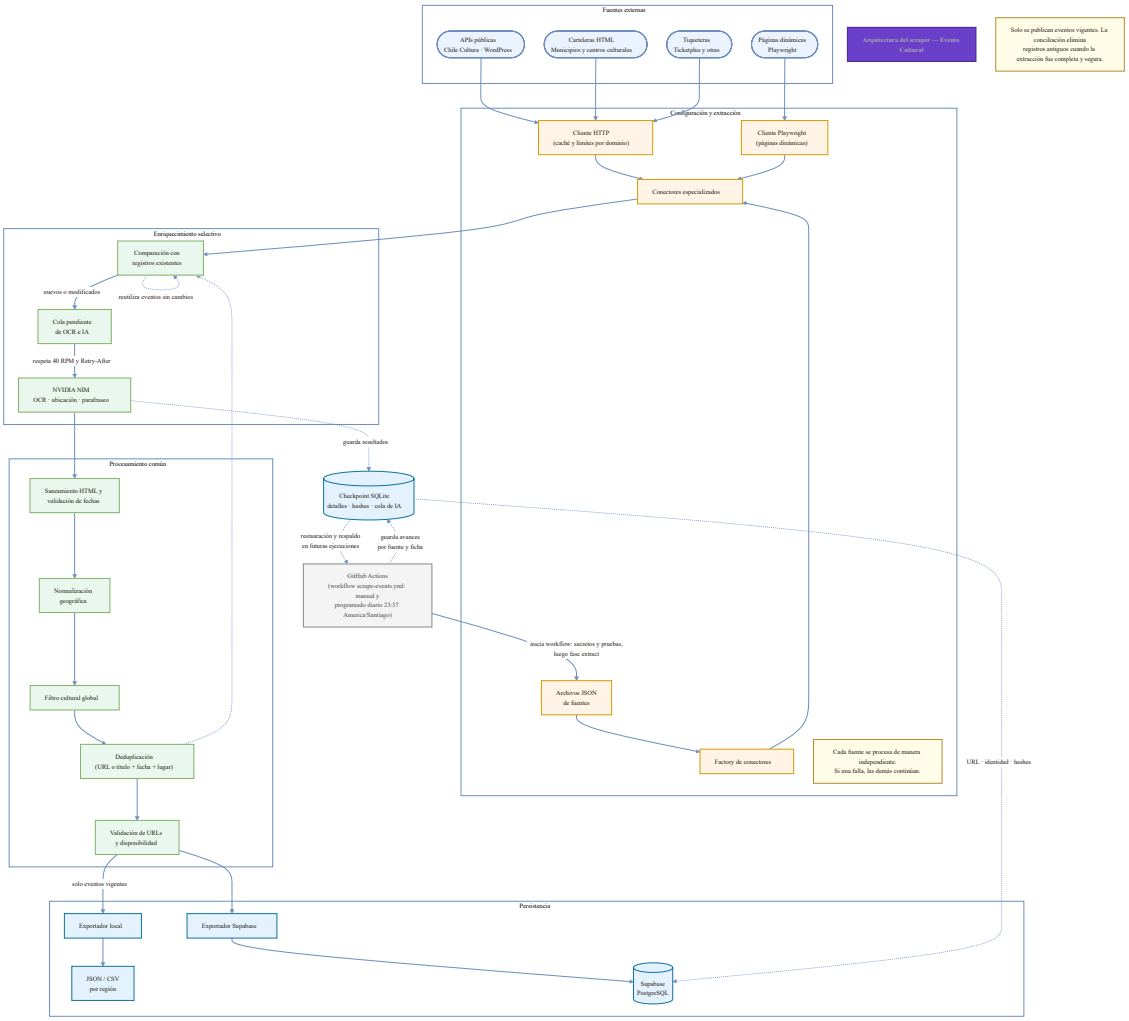


Figura 2: Arquitectura del scraper (original del equipo, fuente editable: `Diagramas/07_arquitectura_scraper.mmd`, Mermaid).

3.2. Conectores y fuentes

Las fuentes se declaran en `sources/*.json`: 21 permanentes más 1 estacional de septiembre (FondasChile), con plantillas por familia en `sources/templates/`.

Conector	Uso
`wordpress_rest`	APIs WordPress REST con `field_map` declarativo
`json_api`	APIs JSON genéricas con paginación
`json_ld`	Sitios con datos estructurados Schema.org
`html_cards`	Carteleras HTML estáticas con selectores CSS declarativos
`eventon`, `tribe`	Plugins de eventos (EventON, The Events Calendar)
`ticketplus`, `maipu`, `fisa`	Conectores especializados
`generic`	Busca JSON-LD primero; usa NVIDIA NIM solo si faltan datos estructurados

Cuadro 2: Conectores disponibles en el paquete `eventos/connectors/`.

3.3. Enriquecimiento selectivo con NVIDIA NIM

Solo los eventos nuevos o modificados pasan por OCR, normalización de ubicación y redacción editorial; los eventos sin cambios se reutilizan por hash de entrada. Las tareas pendientes quedan en la cola persistente del checkpoint SQLite y se recuperan en ejecuciones futuras. El limitador global `NVIDIA_RPM_LIMIT=40` espacia las peticiones aunque varios conectores terminen simultáneamente.

3.4. Pipeline reanudable y concurrencia

Cada fuente y resultado de IA se guarda en `state/pipeline.sqlite3`. El workflow de GitHub Actions restaura y conserva este estado mediante artefactos. Con `SOURCE_WORKERS=4` se crean trabajadores internos; las fuentes de un mismo dominio se procesan en serie (*bulkhead* por dominio) y la salida es determinista. La caché HTTP condicional persistente (ETag/Last-Modified) evita descargas innecesarias.

3.5. Patrones de diseño aplicados

Strategy (conectores), Registry/Factory, Repository, inyección de dependencias para pruebas, Adapter (`SupabaseExporter`), Chain of Responsibility (ubicación), Rate Limiter global (NVIDIA), Bulkhead por dominio, Cache-Aside condicional y post-processing pipeline obligatorio de saneamiento y vigencia (un conector nuevo no puede saltárselo).

3.6. Pendientes de la arquitectura del scraper

- La periodicidad del workflow quedó resuelta: el bloque `schedule` está activo desde el 24-09-2026 y, desde el 29-09-2026, programa la ejecución diaria a las 23:37 (America/Santiago) del workflow "Extraer eventos y sincronizar Supabase". Cada ejecución programada restaura el checkpoint previo, verifica los secretos, ejecuta la suite de pruebas, extrae conservando checkpoints como artefactos (retención de 30 días), enriquece los pendientes y sincroniza con Supabase. La prueba dependiente del reloj fue corregida el 29-09-2026 (commit `d201b0a`: reloj fijado en la prueba de eventos CEINA) y la corrida manual #13 finalizó en verde. **Pendiente de completar por el equipo:** primera ejecución automática completa con la suite en verde.
- Las migraciones SQL del modelo (carpeta `supabase/`) no se incluyeron en la copia local entregada, pero están disponibles en el repositorio oficial del proyecto en GitHub (`https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales`), junto con el workflow `scrape-events.yml` en ejecución.

4. Arquitectura del frontend

4.1. Stack real

Componente	Versión
Angular	22.0.1 (componentes standalone, sin NgModules)
Ionic Framework	^9.0.0
Capacitor	8.x (app, haptics, keyboard, status-bar)
Leaflet + OpenStreetMap	mapa interactivo real (tiles públicos, sin API key)
TypeScript	~6.0.0
Pruebas	Vitest ^4.0.8 (con jsdom)
Lint	ESLint ^9.16.0 + angular-eslint 22.0.0

Cuadro 3: Stack del frontend según `package.json`.

4.2. Estructura y navegación

La aplicación organiza el código en `core/` (modelos `Evento` y `Recomendacion`; servicios con signals: `EventosService`, `RecomendacionService`, `ToastService` y `ViewPreferencesService`), `features/home/` (descubrimiento y layouts de inicio), `layout/` (`MainLayoutComponent`) y `pages/`. Las rutas registradas (`app.routes.ts`) son: `/home`, `/para-ti`, `/calendario`, `/mapa`, `/evento/:id`, `/publicar`, `/perfil` y `/login`, con redirecciones de compatibilidad (`/buscar` y `/favoritos`) y la página de regreso OAuth (`/auth/callback`).

La navegación es adaptativa sin JavaScript adicional: tab bar inferior en móvil y barra superior en escritorio, ambas dentro de `MainLayoutComponent` mediante las clases responsive de Ionic. La identidad visual (paleta terracota/verde/ámbar y tipografías Fraunces/Inter) está centralizada en `src/theme/variables.scss`.

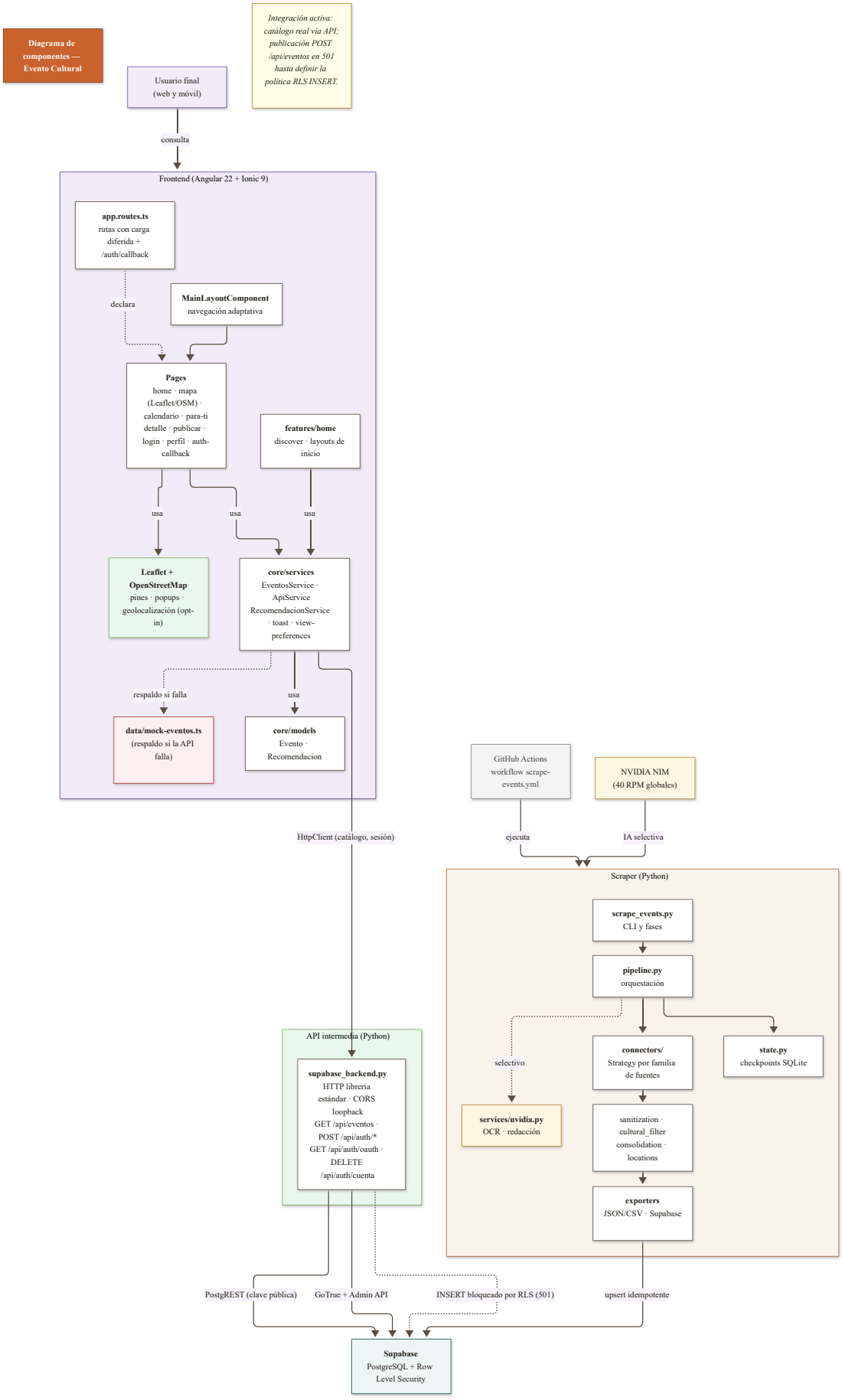


Figura 3: Diagrama de componentes del sistema (fuente editable: `Diagramas/02_componentes.mmd`, Mermaid).

4.3. Filtros y funcionalidad

`EventosService` implementa orden por fecha, nombre, categoría, cercanía y actualización, y filtros combinables: texto, categorías múltiples, región, comuna, rango de fechas, gratuito, modalidad (presencial/virtual/híbrido), público objetivo y accesibilidad. La vista "para ti" (`RecomendacionService`) sustituyó a favoritos como experiencia de recomendación.

El catálogo ya no es estático: `EventosService.cargarEventos()` lo obtiene directamente desde Supabase con `@supabase/supabase-js` (`SupabaseService.catalogo()`, tabla `events` paginada junto con comunas y categorías) al arrancar la aplicación y lo traduce al modelo interno (incluyendo coordenadas lat/lng para el mapa interactivo con Leaflet/OSM). `ApiService` (como fachada reactiva sobre `SupabaseService`) gestiona además la sesión real del usuario (Supabase Auth vía GoTrue: login y registro con correo y contraseña, flujo OAuth Google preparado con regreso en `/auth/callback` y eliminación de cuenta mediante la Edge Function `admin-account`), manteniendo el estado en signals reactivas del framework.

4.4. Estrategia de despliegue

Decisión del equipo (comunicada el 24-09-2026), consistente con el stack definido en Fase 1:

- **Scraper:** automatización con **GitHub Actions** (workflow `scrape-events.yml`, ejecutándose como job programado).
- **Web:** el despliegue en **Vercel** (hosting previsto en el design brief de Fase 1) representa la plataforma objetivo para disponibilizar la aplicación web en producción: dado que el frontend ahora se conecta directamente a Supabase mediante la clave pública anon protegida por RLS y funciones Edge, la aplicación es totalmente estática/JAMstack y apta para despliegue inmediato en Vercel, restando únicamente la definición final de la política RLS INSERT para la publicación abierta de eventos por organizadores.
- **Android (diferido):** aunque la compilación con **Capacitor 8** (dependencias presentes en `package.json`) y **Android Studio** está definida como estrategia, la generación del APK se realizará cuando la web esté completamente estabilizada; en esta fase no se construirá la app móvil (decisión del equipo).
- **No se utilizará Docker** como mecanismo de despliegue (ver Sección 9).

4.5. Pendientes del frontend

La conexión a Supabase y la autenticación están implementadas en la versión entregada (ver Sección 2). **Pendiente de completar por el equipo:** persistencia de las señales de interés del usuario en `user_event_interactions` (hoy sólo en memoria), política RLS INSERT que habilite la publicación por organizadores y activación de los proveedores OAuth en Supabase Dashboard (el flujo está implementado y probado hasta la pantalla de consentimiento).

5. Modelo de datos (Supabase / PostgreSQL)

5.1. Fuente del modelo

El modelo se documenta a partir de la documentación de base de datos incluida en el entregable del scraper de Fase 2, el DBML editable, el diccionario de tablas (DOCX, versión documentada al 03-10-2026) y el diagrama ER de Fase 1 (`DB_evento_cultural.pdf`). Las migraciones SQL no se incluyeron en la copia local entregada, pero están disponibles en el repositorio oficial del proyecto (`https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales`).

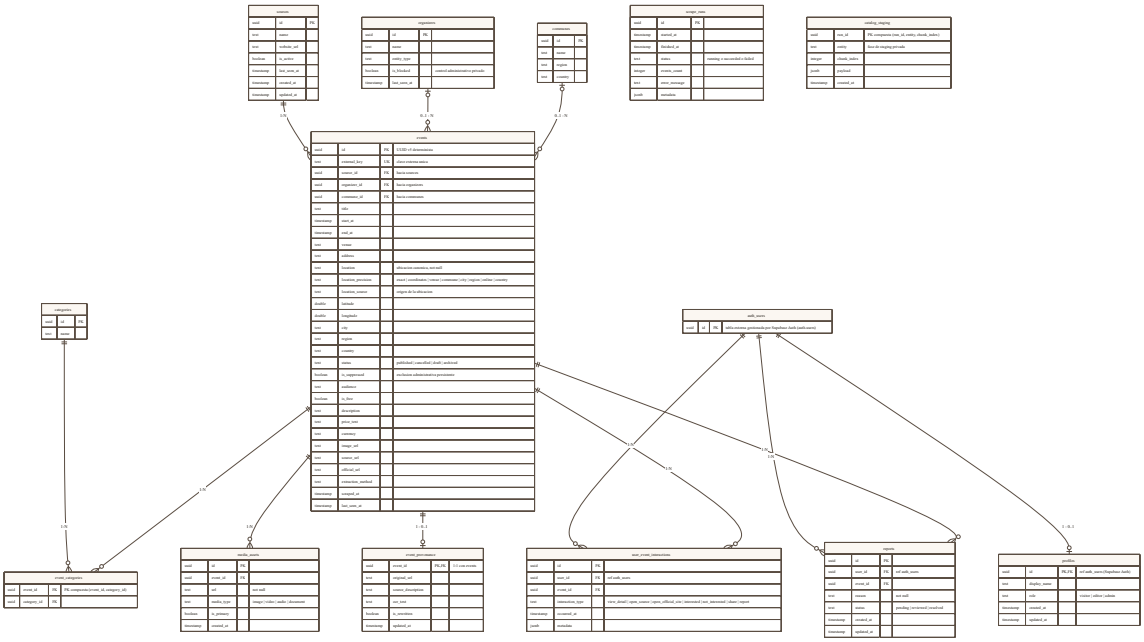


Figura 4: Modelo entidad–relación (fuente editable: `Diagramas/06_modelo_er.mmd`, Mermaid).

5.2. Tablas y finalidad

Los identificadores de eventos, organizadores, comunas y categorías son **UUID v5 deterministas** derivados de claves externas: ejecutar dos veces el mismo scraping actualiza las filas (upsert) sin crear duplicados. El campo `categories` se normaliza en `categories` y `event_categories`, conservando la relación muchos-a-muchos.

Grupo	Tabla	Finalidad
Ingesta y catálogo	`sources`	Agenda, municipalidad, ticketera o sitio de origen
	`organizers`	Institución responsable del evento
	`communes`	Ubicación administrativa normalizada
	`categories`	Etiquetas temáticas, de formato o audiencia
	`events`	Registro canónico del evento consolidado (37 columnas)
	`event_categories`	Relación N:M entre eventos y categorías
	`media_assets`	Imágenes y futuros multimedia
	`scrape_runs`	Bitácora de ejecuciones locales y de GitHub Actions
	`event_provenance`	Evidencia privada: URL original, descripción de la fuente, texto OCR y marca de reescritura
Aplicación	`profiles`, `user_event_interactions`, `reports`	Base prevista de la aplicación; no intervienen en el scraper; `profiles` referencia a `auth_users`, la tabla externa de Supabase Auth
Staging	`catalog_staging`	Área privada de carga por lotes (payload JSON) antes de publicar el catálogo

Cuadro 4: Tablas del modelo de datos según el diccionario y el DBML del proyecto.

5.3. Seguridad (Row Level Security) y ciclo de carga

La migración de RLS habilita Row Level Security en todas las tablas; `scrape_runs` no tiene lectura pública; el usuario no puede cambiar su propio rol ni el estado de sus reportes desde el frontend; la vista `public_event_catalog` hereda las reglas RLS y no expone eventos suprimidos ni organizadores bloqueados.

En el ciclo de carga, la sincronización identifica coincidencias por URL o clave compuesta, carga por lotes (`SUPABASE_BATCH_SIZE`) y registra cada ejecución en `scrape_runs` como `running`, `succeeded` o `failed`. La repetición idempotente es el mecanismo de recuperación ante cargas parciales.

5.4. Limitaciones declaradas

Sin marca explícita de eventos de día completo; `price_text` no permite comparación numérica; categorías sin jerarquía almacenada; registros de Chile Cultura que pueden ser directorios y no eventos fechados; etiqueta inválida conocida (`Calendar_Month Inscribirse`); borrado físico de vencidos solo con `SUPABASE_DELETE_EXPIRED=true`.

6. Diagramas UML

Los cuatro diagramas UML exigidos por el anexo metodológico se escriben como código fuente Mermaid (`.mmd`) y se renderizan a PDF con `mermaid-cli`, en la carpeta `Diagramas/` de esta misma sección. Los casos de uso se derivan de la matriz MoSCoW de Fase 1 y de las páginas realmente implementadas en el frontend. Todos los diagramas (01–07) comparten un estándar visual único (paleta institucional, tipografía Arial, flechas reforzadas), declarado dentro de cada archivo `.mmd`, y fueron regenerados el 27-09-2026 al migrar de Draw.io a Mermaid para automatizar el trazado de líneas y facilitar el mantenimiento; el diagrama entidad–relación (06) fue corregido y reexportado

el 03-10-2026 para alinearlo con el diccionario de tablas y el DBML vigentes; los diagramas 01 (arquitectura general), 02 (componentes) y 04 (secuencia) fueron regenerados el 10-10-2026 para reflejar la integración del sistema: la API intermedia (`supabase_backend.py`) entre el frontend y Supabase, la autenticación real con regreso OAuth en `/auth/callback` y el mapa interactivo (Leaflet sobre OpenStreetMap).

6.1. Casos de uso

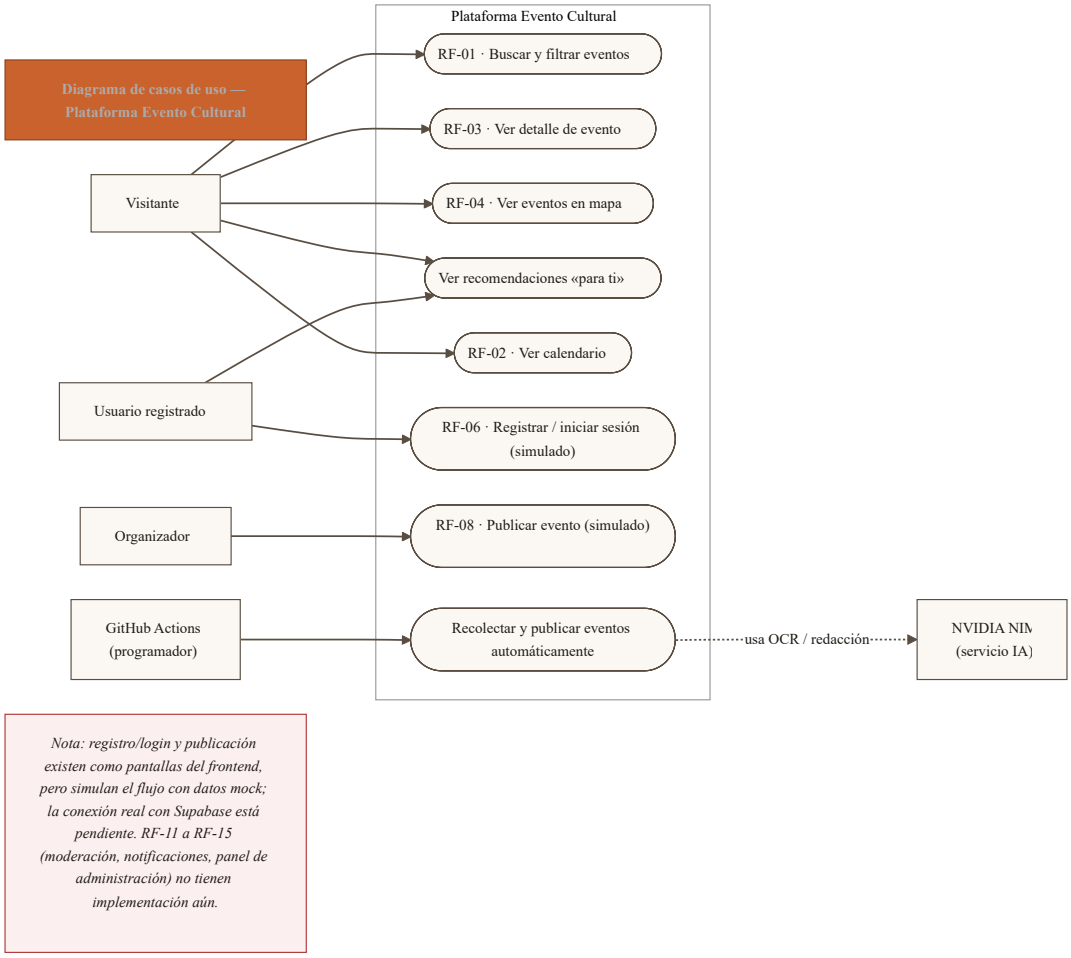


Figura 5: Casos de uso de la plataforma (fuente editable: `Diagramas/03_casos_de_uso.mmd`, Mermaid).

El visitante busca, filtra, ve el detalle, consulta el mapa interactivo real (Leaflet sobre OpenStreetMap) y el calendario; el usuario registrado inicia sesión con correo y contraseña (Supabase Auth vía `supabase-js`; OAuth Google preparado) y recibe recomendaciones; el organizador intenta publicar eventos, pero el INSERT queda sujeto a la política RLS INSERT (aún no definida); GitHub Actions activa la recolección automática, que usa NVIDIA NIM como servicio externo de OCR y redacción.

6.2. Secuencia de la funcionalidad principal

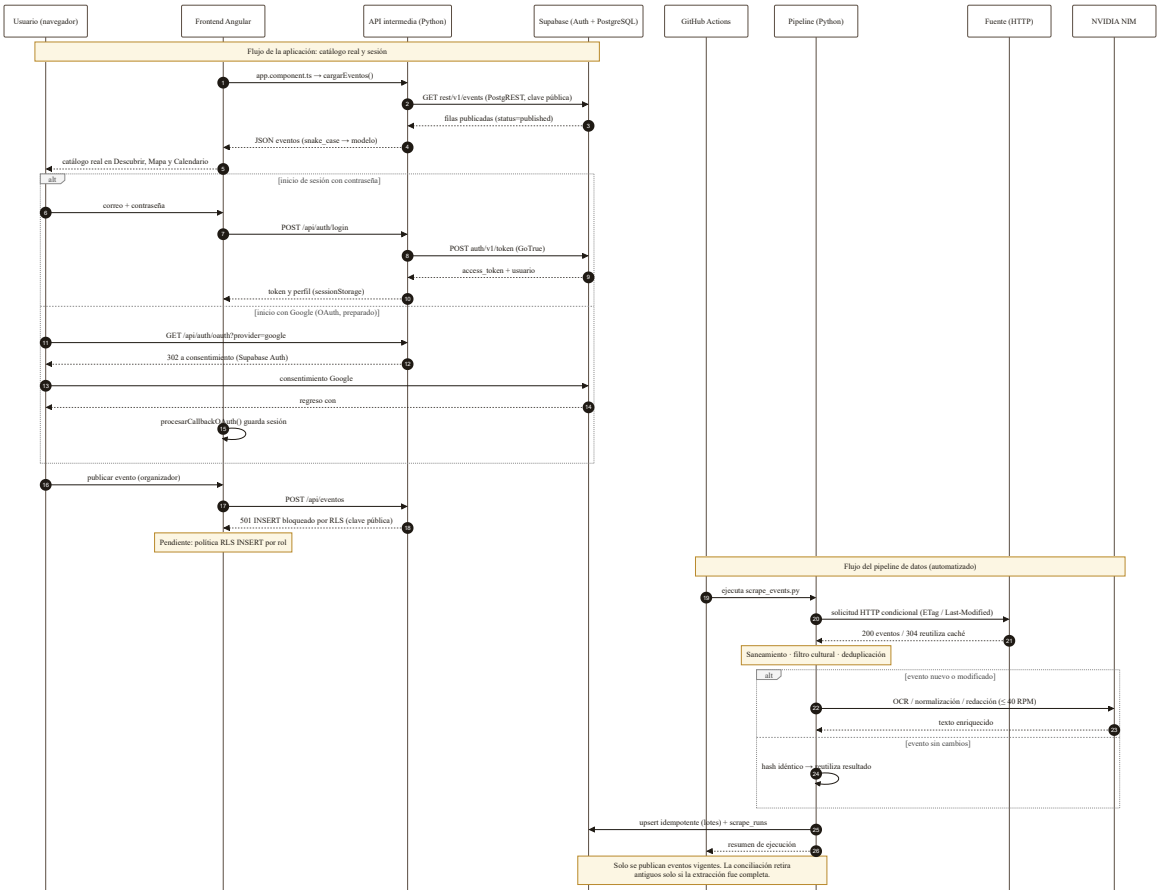


Figura 6: Secuencia: extracción, enriquecimiento y publicación (editable: `Diagramas/04_secuencia.mmd`, Mermaid).

El flujo principal del sistema es la obtención y publicación de eventos: GitHub Actions ejecuta el pipeline; este restaura el checkpoint, consulta las fuentes con caché condicional, sanea y dedu-

plica, decide mediante hash qué eventos pasan por IA (con límite de 40 RPM), reutiliza resultados previos y sincroniza Supabase con upserts idempotentes, registrando la corrida en `scrape_runs`.

6.3. Componentes y clases

El diagrama de componentes (Figura 3) muestra los módulos del frontend y del scraper y su comunicación con la base de datos. En el frontend destacan además el mapa interactivo real (Leaflet sobre tiles de OpenStreetMap, con pines ligados al catálogo, popups y geolocalización opcional del usuario) y la página de regreso OAuth (/auth/callback) integrada en el router. El diagrama de clases (conceptual) refleja los roles reales del núcleo del scraper verificados en el entregable de Fase 2: `EventPipeline`, `ConnectorFactory`, `Connector` (Strategy), `Event`, `EventSanitizer`, `LocationNormalizer`, `NvidiaService`, `SupabaseExporter`, `DatasetExporter` y `PipelineState`.

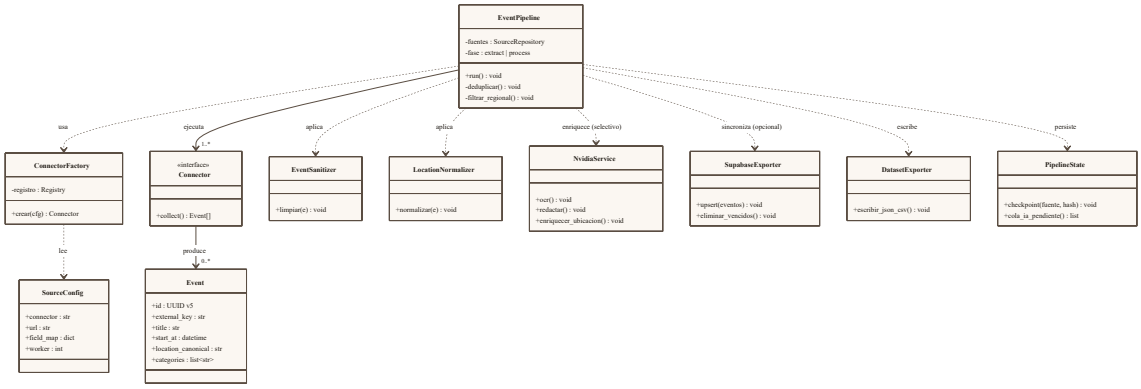


Figura 7: Diagrama de clases conceptual del núcleo del scraper (editable: `Diagramas/05_clases.mmd`, Mermaid).

6.4. Pendientes de UML

- **Resuelto (10-10-2026):** los diagramas 01, 02 y 04 fueron regenerados desde sus fuentes `.mmd` editables para mostrar la arquitectura vigente (API intermedia, autenticación real, mapa interactivo), eliminando la inconsistencia *texto / figura* que existía.
- Si el docente exige notación UML estricta: los diagramas de clases (05) y casos de uso (03) son versiones conceptuales en Mermaid; una regeneración formal añadiría atributos y multiplicidades por clase, y relaciones «include»/«extend» con actores delimitados. La fuente editable (`Diagramas/*.mmd`) es el punto de partida.
- Diagrama de clases del frontend (opcional): complementar el (05) con los servicios Angular (`EventosService`, `ApiService`, `RecomendacionService`, `ViewPreferencesService`). **Pendiente de completar por el equipo:** confirmar la exigencia del docente, definición de responsable y fecha.

7. Requisitos no funcionales

7.1. Requisitos definidos en Fase 1 (MoSCoW)

7.2. Estado real por requisito

ID	Categoría	Requerimiento	Prioridad
RNF-01	Usabilidad	Interfaz clara para ciudadanos y organizadores	Must
RNF-02	Compatibilidad	Diseño responsive / multiplataforma	Must
RNF-03	Rendimiento	Tiempos de carga aceptables con alto volumen	Should
RNF-04	Seguridad	Protección de datos en tránsito y reposo	Must
RNF-05	Seguridad	Control de acceso por rol	Must
RNF-06	Disponibilidad	Disponibilidad estable según hosting definido	Should
RNF-07	Escalabilidad	Crecimiento sostenido de datos	Should
RNF-08	Calidad de datos	Normalización y consistencia entre fuentes	Must
RNF-09	Mantenibilidad	Código mantenible y documentado	Should
RNF-10	Localización	Contenido en español y contexto chileno	Must
RNF-11	Portabilidad	Despliegue reproducible (GitHub Actions)	Could
RNF-12	Accesibilidad	Buenas prácticas de accesibilidad web	Could

Cuadro 5: Requisitos no funcionales priorizados (matriz MoSCoW de Fase 1).

ID	Estado	Evidencia verificable
RNF-01	Parcial	Navegación adaptativa implementada; sin evaluación de usabilidad registrada. **Pendiente.**
RNF-02	Cumplido	Ionic/Angular responsive; navegación dual móvil/escritorio (`MainLayoutComponent`), grid con breakpoints.
RNF-03	Parcial	Caché condicional, bulkhead por dominio y checkpoints optimizan la extracción; falta medición de tiempos del frontend con datos reales. **Pendiente.**
RNF-04	Parcial	RLS en todas las tablas y secretos solo en entorno/GitHub Secrets/Edge Function; autenticación real implementada en la web (Supabase Auth vía supabase-js: login/registro por contraseña, OAuth preparado, eliminación de cuenta por Edge Function). Falta activar los proveedores OAuth y recuperar-contraseña.
RNF-05	Parcial	El modelo prevé roles (`profiles.role`) y reglas RLS; el inicio de sesión real ya distingue al usuario autenticado en la interfaz (perfil, cierre y eliminación de cuenta), pero faltan la política RLS INSERT que habilite la publicación por rol y el panel de administración.
RNF-06	Parcial	GitHub Actions con ejecución manual y programada diaria a las 23:37 (America/Santiago) desde el 29-09-2026 (cron activo desde el 24-09; corridas #9–#13 del 25 al 29-09; #13 en verde tras corregir la prueba dependiente del tiempo). El despliegue web (Vercel) y la app móvil (Capacitor) quedaron diferidos por decisión del equipo, ver Sección 4. **Pendiente:** primera corrida programada completa en verde (Plan de pruebas, sección 5.1).
RNF-07	Cumplido (diseño)	Paginación por página en conectores, upsert idempotente y SQLite como buffer.

ID	Estado	Evidencia verificable
RNF-08	Cumplido (pipeline)	Sanitización, filtro cultural, normalización de ubicación, deduplicación y UUID deterministas.
RNF-09	Cumplido	Documentación interna del entregable de Fase 2, patrones documentados y 14 módulos de pruebas.
RNF-10	Cumplido	Contenido en español; división administrativa chilena (regiones/comunas).
RNF-11	Parcial	Portabilidad adoptada por el equipo: GitHub Actions (scraper), Vercel (web) y Capacitor 8 + Android Studio (Android); sin Docker por decisión del equipo (ver Sección 9).
RNF-12	Parcial	Accesibilidad como criterio de filtro en el frontend; auditoría formal pendiente. **Pendiente.**

Cuadro 6: Estado real de cada requisito no funcional.

7.3. Requisitos declarados y no implementados

Para evitar inconsistencias en la evaluación se declara explícitamente: la autenticación real ya está implementada en la web (ver Sección 2); las notificaciones (RF-13), el panel de administración (RF-14) y la moderación (RF-11/RF-12 del catálogo funcional) no tienen implementación en el código entregado; las estadísticas (RF-17) quedaron *Won't have* en Fase 1; Docker no está implementado y el equipo decidió no utilizarlo (Sección 9).

7.4. Limitaciones técnicas del desarrollo

El desarrollo se realiza en un contexto académico sin presupuesto, lo que impone dos limitaciones operativas visibles en el diseño del sistema:

- **Cuota de la capa gratuita de IA (principalmente tiempo):** el enriquecimiento con IA se apoya en **NVIDIA Build**, plataforma de prototipado con modelos en la nube de uso gratuito pero sujeto a una cuota global de **40 peticiones por minuto (RPM)**. Esta restricción de tiempo condiciona el volumen de eventos que pueden procesarse por corrida y explica dos decisiones de diseño ya documentadas: la IA se aplica *selectivamente* (solo a eventos nuevos o modificados, con control de costo y latencia) y el pipeline es *reanudable* (checkpoints SQLite por fuente y ficha, que evitan repetir trabajo tras un corte o una interrupción por cuota).
- **Sin infraestructura propia de cómputo (hardware):** el equipo no dispone de un servidor donde alojar un modelo de IA especializado (OCR de afiches y análisis de texto); por eso se utiliza NVIDIA Build como alternativa gratuita de prototipado en la nube en lugar de un despliegue local o dedicado.

Otras limitaciones visibles del estado actual —sesiones y señales de interés sin persistencia en base de datos (`user_event_interactions`), publicación bloqueada por RLS hasta definir su política, proveedores OAuth aún no activados y despliegue web aún no efectuado— no son restricciones de infraestructura sino trabajo pendiente ya declarado, trazado en las secciones de estado real y de pruebas. La ejecución programada del scraper está activa (cron diario a las 23:37 America/Santiago desde el 29-09-2026, activo desde el 24-09); la corrección de la prueba dependiente del tiempo (29-09) dejó la corrida manual #13 en verde y resta confirmar la primera corrida programada completa, documentada en el plan de pruebas.

Pendiente de completar por el equipo.decisión futura sobre migrar el enriquecimiento con IA a infraestructura propia o a una cuota pagada; en tal caso, las restricciones de esta subsección y las decisiones de diseño asociadas se reevaluarán.

8. Pruebas

8.1. Pruebas existentes del scraper

Comando oficial (README del scraper): `python -m unittest discover -s tests -v`. La carpeta `tests/` contiene 14 módulos (aproximadamente 1.900 líneas) con carácter unitario y de módulo: usan dobles de prueba del cliente HTTP, del repositorio de fuentes y del exportador, según la inyección de dependencias documentada.

Módulo	Qué valida
`test_core.py`	Configuración, mapeo WordPress, deduplicación/filtro y exportación
`test_event_sources.py`	Conectores (Fisa, EventOn, Generic, Ticketplus, HtmlCards) y catálogo mínimo de 10 fuentes activas
`test_supabase.py`	Transformación idempotente al esquema de Supabase
`test_resumable_pipeline.py`	Checkpoints, reanudación y pausas NVIDIA
`test_performance.py`	Caché condicional, lecturas Maipú, archivos operativos y pipeline paralelo
`test_locations.py`	Normalización de ubicación (reglas y NVIDIA)
`test_content_enrichment.py`	Enriquecimiento editorial
`test_nvidia.py`	Respuestas y límites NVIDIA
`test_sanitization.py`	Limpieza HTML y vigencia
`test_cultural_filter.py`	Filtro de pertinencia cultural
`test_jsonld.py`	Extracción JSON-LD
`test_availability.py`	Disponibilidad de eventos
`test_analytics.py`	Métricas operativas
`test_source_validation.py`	Reglas del validador de fuentes

Cuadro 7: Pruebas unitarias y de módulo existentes en el scraper.

8.2. Pruebas existentes del frontend

El frontend configura Vitest con jsdom (comando `ng test`) e incluye pruebas de humo de creación de componentes y servicios. No cubre todavía la lógica de filtros ni de servicios.

8.3. Pruebas pendientes

8.4. Registro de resultados

9. Docker: estado actual y plan

9.1. Estado real verificado

Conclusión: Docker no está implementado en ninguna parte del sistema (se declara explícitamente porque el anexo metodológico exige documentar su estado). La estrategia de despliegue adoptada por el equipo (decisión comunicada el 24-09-2026) no contempla contenedores:

- **Scraper:** ejecución local en Windows (`instalar.bat`, `ejecutar.bat`) y automatización en **GitHub Actions**; el workflow instala por sí mismo Python, Playwright y Chromium.
- **Web:** despliegue en **Vercel** (hosting previsto en el design brief de Fase 1 y confirmado por el equipo); build de producción con `ng build`. A la fecha no existe evidencia de un despliegue ejecutado.

#	Prueba	Alcance	Tipo
1	Unitarias de `EventosService` (filtros y orden)	Frontend	Unitaria
2	Unitarias de recomendaciones y preferencias	Frontend	Unitaria
3	Integración frontend–Supabase (vista pública)	Frontend + BD	Integración
4	Integración scraper–Supabase (upsert idempotente)	Scraper + BD	Integración
5	Rendimiento: carga del listado con volumen real	Frontend	Rendimiento
6	Rendimiento: pipeline completo (usar `run_summary.json`)	Scraper	Rendimiento
7	Seguridad: verificación de políticas RLS por rol	BD	Seguridad
8	Seguridad: saneamiento de entradas de usuario	Scraper/Frontend	Seguridad
9	Accesibilidad: auditoría Lighthouse/axe	Frontend	No funcional
10	Registro de resultados y trazabilidad	Proceso	Proceso

Cuadro 8: Plan de pruebas pendiente. Responsables, fechas y criterios de aceptación: **pendiente de completar por el equipo**.

Fecha	Suite ejecutada	Resultado	Responsable
—	—	**Pendiente de completar por el equipo**	—

Cuadro 9: Registro de ejecución de pruebas (sin resultados inventados).

- **Android:** compilación de la aplicación con **Capacitor 8** y **Android Studio** (dependencias de Capacitor presentes en `package.json`).
- **Base de datos:** servicio administrado de Supabase, sin contenedores.

9.2. Plan de implementación (alternativa documentada, no adoptada)

El equipo decidió no utilizar Docker en la estructura de despliegue actual. El siguiente plan se conserva documentado como alternativa futura y como respuesta al punto 5 del anexo metodológico; su eventual implementación queda **pendiente de completar por el equipo** solo si la estrategia de despliegue cambiara.

1. **Contenedor del scraper:** imagen base `python:3.12-slim`; instalación de `requirements.txt` y Chromium de Playwright dentro de la imagen; volúmenes para `state/` (checkpoints) y `data/` (respaldos); configuración por variables de entorno según `.env.example`, sin credenciales embebidas; ejecución por fases (`--phase extract` / `--phase process`).
2. **Contenedor del frontend:** build multi-etapa (etapa Node con `ng build`; etapa de servicio estático con nginx), con variables de entorno de runtime para el endpoint del backend futuro.
3. **Composición opcional:** `docker-compose.yml` de desarrollo para el frontend servido y el scraper como job;la base de datos permanece como servicio administrado de Supabase.

Componente	Dockerfile	docker-compose	Contenedores en ejecución
Scraper (Python)	No	No	No
Frontend (Angular/Ionic)	No	No	No
Base de datos	— (servicio administrado Supabase)	No	No

Cuadro 10: Estado de Docker verificado por inspección de ambos entregables: no implementado.

Criterios de aceptación propuestos: (1) `docker build` de ambas imágenes sin credenciales; (2) una ejecución completa del scraper en contenedor produce `data/events.json` y registra la corrida en `scrape_runs`; (3) el frontend en contenedor responde en el puerto definido; (4) README actualizado con las instrucciones `docker compose up`.

10. Innovación

10.1. ¿Qué problema resuelve?

En Chile la oferta de eventos culturales está fragmentada en decenas de publicadores independientes (municipalidades, centros culturales, ticketeras, universidades y portales regionales), cada uno con formatos, fechas y estructuras distintos: agendas WordPress, APIs, carteleras HTML e imágenes de afiches. El ciudadano debe revisar sitio por sitio y muchos eventos —especialmente municipales y gratuitos— no llegan a las plataformas comerciales. La magnitud del problema quedó documentada en el propio proyecto: el inventario de fuentes levantó 390 candidatas, de las cuales se integraron 21 permanentes más 1 estacional mediante conectores específicos.

10.2. ¿Qué hace diferente a la solución?

- **Cobertura municipal y regional:** prioriza agendas culturales públicas (municipios, centros culturales, universidades), no solo ticketeras comerciales (tabla de 22 fuentes configuradas en el entregable del scraper de Fase 2).
- **Enriquecimiento con IA selectivo:** OCR de afiches, normalización de ubicaciones y redacción editorial con NVIDIA NIM, aplicado solo a eventos nuevos o modificados, con control de costo y latencia (40 RPM globales).
- **Calidad de datos como regla, no como tarea:** sanitización, filtro de pertinencia cultural, normalización geográfica y deduplicación obligatorias para toda fuente; un conector nuevo no puede saltárselos.
- **Pipeline reanudable y auditable:** checkpoints SQLite por fuente y ficha; reanudación tras cortes y bitácora en `scrape_runs`.
- **Publicación siempre vigente:** reutiliza eventos sin cambios y retira registros antiguos solo cuando la extracción fue completa y segura.
- **Recomendación con señales de interés:** vista "para ti" en lugar de un listado estático, y experiencia multiplataforma desde el mismo código.

10.3. ¿Qué valor agrega?

Para el ciudadano, un solo lugar para descubrir eventos reales (catálogo en Supabase, sincronizado por el pipeline) con búsqueda, filtros (región, comuna, fecha, gratuito, modalidad, accesibilidad), mapa interactivo (Leaflet sobre OpenStreetMap, con geolocalización) y calendario. Para los organizadores públicos, visibilidad sin costo para agendas municipales y de centros culturales. Para

el ecosistema de datos, un catálogo normalizado con procedencia verificable (`event_provenance` conserva texto original y OCR). Para lo académico, la aplicación real de IA generativa en un pipeline de datos con control de costos y de patrones de arquitectura documentados y probados.

Declaración de honestidad: los resultados de operación reales (número de eventos publicados, tiempos, usuarios) no se afirman en este documento y deben registrarse con datos de ejecución cuando el equipo los genere.

10.4. Modelo de sostenibilidad y proyección de negocio (propuesta futura; no implementada en el MVP)

Estado actual del MVP. El proyecto no implementa monetización, pagos, planes comerciales, venta de entradas, cuentas de organizadores, publicidad, patrocinios, enlaces de afiliación ni contratos con entidades. El MVP funciona como un **catálogo cultural gratuito**: obtiene información desde fuentes de acceso público, conserva la procedencia y trazabilidad de los datos (`event_provenance`) y redirige al usuario hacia la fuente original de cada evento para consultar detalles, reservar o comprar entradas cuando corresponda. La propuesta de valor actual consiste en centralizar el descubrimiento de eventos culturales, facilitar búsquedas y filtros, preservar la trazabilidad de las fuentes y aumentar la visibilidad de actividades culturales sin cobrar a usuarios ni organizadores por la información recopilada.

Proyección futura de sostenibilidad. Como posibilidad futura, condicionada y no prioritaria, podría evaluarse una opción de visibilidad mejorada para organizadores verificados (eventos destacados o espacios promocionales). Esta alternativa solo podría examinarse cuando existan, de manera simultánea: audiencia suficiente, métricas confiables, perfiles de organizadores verificados, mecanismos de contacto comercial, capacidad de moderación y validación legal, económica y ética. Si alguna vez se implementara contenido destacado o patrocinado, debería (i) identificarse explícitamente como destacado, promocionado o patrocinado; (ii) mantenerse separado de las recomendaciones orgánicas basadas en intereses del usuario; (iii) no impedir el acceso gratuito al descubrimiento de eventos; (iv) respetar la procedencia de la información y las condiciones de uso de las fuentes; y (v) no alterar silenciosamente los resultados ni favorecer contenido pagado sin transparencia.

Alternativas descartadas para la primera etapa. No se implementarán inicialmente: la comisión por venta de entradas, la intermediación de pagos, los planes de suscripción para organizadores, la venta o explotación comercial de datos personales y la publicidad invasiva (enlaces de afiliación o promoción que modifique silenciosamente las recomendaciones). La venta intermediada de entradas exigiría integración con medios de pago, acuerdos comerciales, gestión de reembolsos, soporte, obligaciones tributarias, seguridad, protección de datos y responsabilidades legales que exceden el alcance técnico y académico del MVP. Los planes para organizadores requerirían un portal de gestión, perfiles verificados, publicación controlada, estadísticas reales, atención a usuarios y una base de audiencia suficiente que hoy no existe.

Limitaciones actuales de Vercel. El frontend del MVP se proyecta para desplegarse en Vercel bajo el plan **Hobby**, orientado a uso personal y no comercial. Por ese motivo, durante la etapa actual no se implementarán anuncios, promociones pagadas, patrocinios comerciales, enlaces de afiliación, cobro a organizadores ni procesamiento de pagos. El scraper no se despliega en Vercel: se ejecuta periódicamente mediante GitHub Actions (diario a las 23:37, America/Santiago), y Supabase se utiliza como servicio administrado para la persistencia de datos. Una monetización futura exigiría evaluar previamente una migración a un plan comercial de Vercel o a una infraestructura alternativa compatible con uso comercial, además de las validaciones funcionales, legales, económicas y éticas necesarias.

Ninguna de las líneas anteriores compromete el carácter público y gratuito del descubrimiento de eventos, que es el objetivo central del proyecto.

11. Trazabilidad documental

Cada afirmación de este documento proviene de las siguientes fuentes verificables:

Sección	**Evidencia utilizada**
Arquitectura general, scraper y frontend	Entregables del scraper y frontend incluidos como evidencias de sistema en Fase 2; `package.json`, `app.routes.ts`, `supabase.service.ts`, `api.service.ts`, Edge Functions y diagramas técnicos de Fase 2. Integración frontend–Supabase implementada y verificada en TypeScript (catálogo real directo, autenticación y OAuth preparado).
Modelo de datos	Documentación de base de datos incluida en el entregable del scraper de Fase 2, modelo DBML, diccionario de tablas y `DB_evento_cultural.pdf` (Fase 1).
Diagramas UML	Rutas y servicios reales del frontend; clases y patrones documentados del scraper; matriz MoSCoW de Fase 1. Los siete diagramas (01–07) comparten estándar visual, fueron pulidos y reexportados el 27-09-2026 y el ER (06) fue actualizado el 03-10-2026.
Requisitos no funcionales	Matriz `Requerimientos_Funcionales_NoFuncionales_MoSCoW.xlsx` (Fase 1) y evidencia de código por requisito.
Pruebas	Carpeta `tests/` del scraper (14 módulos, aproximadamente 1.900 líneas), sección “Pruebas” de su README y configuración Vitest del frontend.
Docker	Inspección completa de ambos entregables (sin Dockerfile ni compose) y `CONTENIDO_DEL_REPOSITORIO.md`.
Innovación	Design brief de Fase 1, matriz MoSCoW, tabla de fuentes del README del scraper y funcionalidad “para ti” del frontend.

Cuadro 11: Trazabilidad entre secciones del documento y evidencia del repositorio.

Los cambios y pendientes vigentes se resumen en los índices de documentación de Fase 2.